

4 Access control

The following files are shown by an `ls -l` command on a typical Unix system:

```
-r-xr-sr-x 1 charlie acct 512 08-07-2008 15:35 accounting
-r--r----- 1 alice acct 256 08-07-2008 15:23 accounts
-rwxr-xr-x 1 system system 128 01-01-1970 00:01 editor
-rw-r--r-- 1 alice users 768 08-07-2008 15:26 cv.txt
-r--r----- 1 bob gurus 344 08-07-2008 15:25 exam
-r--r----- 1 alice gurus 188 08-07-2008 15:26 solutions
```

Unix users `alice` and `bob` are both members of the group `users` only, while `charlie` is a member of the group `gurus` only. Application `editor` allows users to read and write files of arbitrary name, whereas application `accounting` only allows users to append data records to the file `accounts`. Draw up an access control matrix with subjects `{alice, bob, charlie}` and objects `{accounts, cv.txt, exam, solutions}` that shows for each combination of subject and object whether the subject is able to read (R), (over)write (W), or at least append (A) records to the respective object.

	accounts	cv.txt	exam	solutions
alice				
bob				
charlie				

Cont. LEGGERE / PERMESSA: PRENDIAMO COME ESEMPIO LA PRIMA RIGA:

- CHARLIE = UTENTE PROPRIETARIO R-X
- ACCT (GROUP) R-S
- TUTTI GLI ALTRI R-X
- X SIGNIFICA "ESECUTORE" OVVERO CHI POSSIEDE QUAL PERMESSA PER ESEGUIRE IL FILE.

	accounts	cv.txt	exam	solutions
alice	R, A	RW		R
bob	A	R	R	-
charlie	A	R	R	R

CHARLIE: A IN QUANTO LO PIU' NEL TESTO. PER CV.TXT IN QUANTO CHARLIE ALLADE NELLA CATEGORIA TUTTI GLI ALTRI. READ SU EXAM IN QUANTO CHARLIE FA PARTE DEL GRUPPO GURUS (SENZA DAL TESTO) E SOLUTION READ SEMPRE PER TUTTI.

ALICE: HA SU ACCOUNT IN QUANTO NEL GRUPPO ACCOUNTS VEDIAMO IL NOME ALICE E CONTE PERMESSA HA A. BISOGNA PER ANCHE READ SU EXAM, NOTIAMO CHE ABBIAMO GURUS MA NON DOBBIAMO TENERE A MENTE. PRENDIAMO IN CONSIDERAZIONE SOLO LA NOSTRA EXAM E ALICE NON APPARE.

BELL-LAPADULA

REGOLE DI LINGUA: PER POTER LEGGERE UN FILE, L'UTENTE DEVE ESSERE "ABBOZZA IN ADO". OGNI UTENTE E OGNI RISORSA HANNO UN LIVELLO TOP SECRET > SECRET > CONFIDENTIAL E UN INSIEME DI CATEGORIE COME AD ESEMPIO {NEO, GREEN, BLUE}.

- LIVELLO: UTENTE $\geq$ RISORSA
- CATEGORIE: L'UTENTE DEVE CONTENERE ALMENO TUTTE LE CATEGORIE DELLA RISORSA

REGOLE DI SCRITTURA: PER POTER SCRIVERE UN FILE, L'UTENTE NON DEVE AVERE "SPERANZA" SECRET VERSO IL BASSO. PER SCRIVERE SOLO VERSO L'ALTO O ALLO STESSO LIVELLO:

- LIVELLO: UTENTE $\leq$ RISORSA
- CATEGORIE: LE CATEGORIE DELL'UTENTE DEVONO ESSERE UN SOTTOINSIEME DI QUELLE DELLA RISORSA

2024 Febbraio - Bell-LaPadula

4. Access Control

Consider the Bell-LaPadula security model. Indicate the permissions granted to a user with the following "security clearances" on a resource with security label (secret, {red, blue}):

1. (top secret, {red, blue}): _____
2. (secret, {red}): _____
3. (secret, {red, black}): _____
4. (secret, {red, blue}): _____
5. (confidential, {red, blue}): _____
6. (confidential, {blue}): _____
7. (top secret, {red, green, blue, black}): _____

1) LINGUA. COME SPESICO SOPRA TOP SECRET E' $\geq$ DI SECRET. SIVIVI PER FORZA LINGUA, INOLTRE CONTIENE ALMENO UNO VOLTA LE RISORSE NEO E BLUE

- 2) SCRITTURA IN QUANTO SEI UNO E NEI 6' UN SOLO RISULTATO DI {NEI, B-I}
- 3) MASSIMA PERMISSIONE
- 4) CANTUM E SCRITTURA
- 5) SCRITTURA
- 6) SCRITTURA
- 7) CANTUM

2023 Gennaio - Bell-LaPadula

4. Access Control

Consider the following security labeling in the Bell-LaPadula model:

Subject	Label	Object	Label
Alice	L4	P1	L4
Bob	L1	P2	L1
Carol	L5	P3	L5
Dave	L2	P4	L2
Eve	L3	P5	L3

The labels follow a complete ordering $L1 > L2 > L3 > L4 > L5$.

- (a) Interpret the labels as security labels in the simplified Bell-LaPadula model. Fill the access column in the following table with the access (read, write) that BLP would give each subject to the corresponding object.

Subject	Object	Access
Alice	P5	
Bob	P3	
Carol	P5	
Dave	P4	
Eve	P1	

- (b) Now consider the case where the labels have categories in addition to the completely ordered levels. We add categories red and blue. The new label assignments are:

Subject	Label	Object	Label
Alice	L4: {red}	P1	L4: {red}
Bob	L1: {red}	P2	L1: {red, blue}
Carol	L5: {red, blue}	P3	L5: {red}
Dave	L2: {red, blue}	P4	L2: {red}
Eve	L3: {blue}	P5	L3: {blue}

Interpret these labels according to the Bell-LaPadula Model. Fill the access column with the access that BLP would give each subject to the corresponding object: read, write.

Subject	Object	Access
Alice	P1	
Bob	P2	
Carol	P2	
Dave	P4	
Eve	P3	

2) L'UTENTE SAREBBE IL SUBJECT, MENTRE LA NASCITA DEDICATA. PENSANDO IN CONSIDERAZIONE A' L'UTENTE. ALICE NELLA TABELLA SUPERIORE HA VALORE L4, PS HA VALORE L3. **ATTENZIONE!!** OVANDI BENE LA L4.

NOTANDO CHE $L4 \leq L3$ PERCHÉ È SOLO SCRITTURA

PERÒ ANCHE (IN ORDINE) SONO:

- CANTUM
- SCRITTURA
- CANTUM/SCRITTURA → SUGGERISCE SOLO CANTUM
- CANTUM

b)

- CANTUM/SCRITTURA → SUGGERISCE ENTRAMBE LE PERMISSIONI
- SCRITTURA
- CANTUM
- CANTUM IN QUESTO CASO COME COMMENTARE A' D, BLUE È IN PIV
- NESSUN PERMESSO

4. Access Control

This is a simplified dump for the `ls -l` shell command in the current folder.

```
-rw-rw-r-- alice alice 1
-rw-rw-r-- bob bob 2
-rw-rw-r-- charlie charlie 3
-rw-rw-r-- bob bob 4
-rw-rw-r-- alice alice append
-rw-rw-r-- bob append append-super
-rw-rw-r-- charlie charlie editor
```

Unix users are `alice`, `bob` and `charlie`.

The `id` command for each user returns:

- `id alice: uid=1000(alice) gid=1000(alice) groups=1000(append),1000(alice)`
- `id bob: uid=1001(bob) gid=1001(bob) groups=1001(bob)`
- `id charlie: uid=1002(charlie) gid=1002(charlie) groups=1002(charlie)`

There are 3 executable files:

- `editor` lets you open a file with Read and Write permissions;
- `append`, as the name suggests, lets you Append a line to a given file;
- `append-super` does the same as `append`.

Draw up an access control matrix with subjects {alice, bob, charlie} and objects {1, 2, 3, 4} that shows for each combination of subject and object whether the subject will be able to read (R), (over)write (W), or at least append records (A) to the respective object.

	1	2	3	4
Alice	R, W, A	R, W, A	R	R, A
Bob	R	R, W, A	R	R, W, A
Charlie	R	R, W	R, W	R

Alice NEL FILE 2 ASSUME A IN QUANTO E' PRESENTE IL PERMESSO RW. SE IN "OTHERS" CI FOSSE STATO R ALORA ALICE AVREBBE DATO CHARLIE APPEND SUPER, SICCOME E' ALLIATO A BOB AVREBBE DATO I PERMESSI RW E DI COMANDARE A.
 Alice NEL FILE 3 HA SOLO R IN QUANTO ALTRA IN OTHERS. SE AVESSE CHIAMATO APPEND SUPER BOB SAREBBE CADUTO SEMPRE IN OTHERS.
 Alice NEL FILE 4 CADE IN R, CHIAMANDO APPEND-SUPER OTTENE DA BOB A.

🔥 File 4 (`-rw-r--r-- 1 bob bob`) - Il vero trucco dell'esercizio!

- **Permessi base:** Alice ricade in "Others" e ha solo `r--`. Ottiene la R. Di base, non può assolutamente scrivere o modificare questo file.
- **Come ottiene la A? Grazie a `append-super` !**
 1. **Può eseguirlo?** L'eseguibile `append-super` ha permessi `-r-sr-xr-- 1 bob appnd`. L'esecuzione è permessa al gruppo `appnd` (`r-x`). Guardando l'output del comando `id`, Alice appartiene al gruppo `appnd` ! Quindi Sì, può lanciarlo.
 2. **Il potere speciale (SUID):** Questo eseguibile ha una `s` al posto della `x` del proprietario (`bob`). Significa che chiunque lo lanci, lo esegue "indossando i panni" del proprietario Bob.
 3. **L'incrocio:** Alice lancia `append-super` spacciandosi temporaneamente per Bob. Bob possiede il file 4 e ha i permessi di scrittura (`rw-`).
 4. **Il limite logico:** Siccome il testo dice che `append-super` permette *solo* di accodare linee, Alice non ottiene la W piena, ma solo la A.
- **Risultato:** Unendo la sua R di base e la A ottenuta "rubando" i permessi di Bob, il risultato è R, A.

NOTA GENERALE:

Ogni slot ha una funzione ben precisa:

- 🗂️ **1° Slot:** Riservato *solo* alla Lettura (r). Se il permesso manca, trovi un `-`.
- 🗂️ **2° Slot:** Riservato *solo* alla Scrittura (w). Se il permesso manca, trovi un `-`.
- 🗂️ **3° Slot:** Riservato *solo* all'Esecuzione (x oppure la speciale s). Se il permesso manca, trovi un `-`.

💡 **Esempi pratici per allenare l'occhio:**

- `rw-` : Il 1° slot ha la `r`, il 2° ha la `w`, il 3° è vuoto (`-`). Risultato: **Lettura e Scrittura**.
- `r-x` : Il 1° slot ha la `r`, il 2° è vuoto (`-`), il 3° ha la `x`. Risultato: **Lettura ed Esecuzione** (niente scrittura).
- `r-s` : Il 1° slot ha la `r`, il 2° è vuoto (`-`), il 3° ha la `s` speciale. Risultato: **Lettura ed Esecuzione speciale SUID** (niente scrittura).

PER QUESTO MOTIVO CHARLIE NON PUO' USARE A:

👤 **1. L'identità di Charlie**
 Guardando l'output del comando `id` nel testo, vediamo che Charlie appartiene esclusivamente al suo gruppo base: `groups=1002(charlie)`. Non fa parte di nessun altro gruppo.

📁 **2. I permessi di `append-super`**
 La riga dell'eseguibile è: `-r-sr-xr-- bob appnd append-super`
 Scomponiamo chi ha il diritto di lanciare il programma (ovvero chi possiede la `x` o la `s`):

- **Proprietario (`bob`):** `r-s` (può eseguire e attiva il trucco del SUID).
- **Gruppo (`appnd`):** `r-x` (i membri di questo gruppo possono eseguire).
- **Altri (Others):** `r--` (gli altri possono *solo* leggere il codice del file, niente esecuzione).

🚫 **3. L'incrocio e il blocco**

- Charlie non è Bob (salta il primo blocco).
- Charlie non fa parte del gruppo `appnd` (salta il secondo blocco). Alice invece era in questo gruppo, per questo lei poteva usarlo!
- Charlie finisce inesorabilmente nel terzo blocco, quello degli "Others".

💡 **Conclusione:**
 Poiché i permessi per gli "Others" su questo programma sono semplicemente `r--`, a Charlie manca la lettera `x` (execute). Il sistema operativo gli impedisce fisicamente di lanciare il programma. Non potendolo avviare, non può ovviamente beneficiare di alcun potere speciale di scrittura o `append`.

2022 Gennaio - DAC (permessi dei file)

4. Access Control

A restaurant processes orders with a FIFO strategy. Orders are stored in file `orders.txt`. Program `enqueue` modifies `orders.txt` by adding an order to the tail of the queue. Program `dequeue` modifies `orders.txt` by deleting the order at the head of the queue.

Define a UNIX-based ACL of the files so to ensure that the following access control policy is enforced by the operating system:

- willie and winona, the waiters, can add new orders, but cannot read nor delete existing orders;
- charlie and cathy, the cooks, can read and delete existing orders, but cannot add new orders nor carry out any other modification to `orders.txt`;
- mario, the maitre, must be able to read the orders (and possibly do other actions).

Please provide your answer by completing the dump of the `ls -l` shell command (assuming that all relevant files are in the current folder):

```
_____ orders.txt
_____ waiters enqueue
_____ cooks dequeue
```

where `cooks` is a group containing `charlie` and `cathy` and `waiters` is a group containing `willie` and `winona`.

Spiegazione passo-passo:

1. Si parte dal verificare chi sono i proprietari dei file, in questo caso si parte da `orders.txt` e si procede per esclusione. I `waiters`, ossia `willie` e `winona`, non possono essere proprietari del file per due ragioni, la prima è che, in UNIX, un gruppo non può essere proprietario di un file e la seconda è che hanno delle restrizioni su cosa possono fare su quel file. Stesso ragionamento vale per i `cooks`, ossia `charlie` e `cathy`. Andando per esclusione l'unico utente che rimane è `mario`, il maitre.

```
- ____ _ ____ mario ____ orders.txt
- ____ _ ____ ____ waiters enqueue
- ____ _ ____ ____ cooks dequeue
```

2. Gli utenti che lanceranno gli eseguibili `enqueue` e `dequeue` dovranno poter modificare `orders.txt` secondo le direttive, ma lo potranno fare solo se avranno i permessi del proprietario di `orders.txt`, ossia `mario`, quindi egli dovrà essere il proprietario degli eseguibili.

```
- ____ _ ____ mario ____ orders.txt
- ____ _ ____ mario waiters enqueue
- ____ _ ____ mario cooks dequeue
```

5. Non sono previsti permessi da dare a qualsiasi utente esterno ai gruppi e diverso dal proprietario. A questo punto c'è da fare una osservazione non banale, i gruppi possono adesso si lanciare gli eseguibili, ma con i loro permessi, ossia da NON proprietari del file `orders.txt`, ergo quando andranno ad aprire e modificare `orders.txt` gli eseguibili falliranno in quanto l'unico che può modificare tale file è `mario`. Serve il `set-userid` abilitato sui due eseguibili in modo che, quando vengono lanciati, essi vengano eseguiti con i permessi del loro proprietario, ossia `mario`.

```
- rW ____ _ mario ____ orders.txt
- rWs ____ x ____ mario waiters enqueue
- rWs ____ x ____ mario cooks dequeue
```

3. Ora si passa a definire i permessi, partendo da quelli del proprietario di un file che sono i più semplici. Di base un proprietario ha tutti i permessi sui suoi files, ma attenzione che i file non eseguibili, come quelli di testo `.txt`, non possono essere eseguiti e il permesso di esecuzione su di essi non ha senso.

```
- rW ____ _ mario ____ orders.txt
- rWx ____ _ mario waiters enqueue
- rWx ____ _ mario cooks dequeue
```

4. Ora si passa ai permessi dei gruppi, i `waiters` devono poter lanciare la `enqueue` e i `cooks` la `dequeue`.

```
- rW ____ _ mario ____ orders.txt
- rWx ____ x ____ mario waiters enqueue
- rWx ____ x ____ mario cooks dequeue
```

🚫 Step 4 (Senza la `s`): Il programma fallisce

Se lasci solo `rWx`, i camerieri possiedono la `x` nel loro blocco e possono avviare `enqueue`. Ma c'è un problema: il programma "enqueue" cerca di aggiungere testo al file `orders.txt`. Il sistema operativo vede che a fare la richiesta è un cameriere, controlla i permessi di `orders.txt` (che permettono la modifica solo a Mario) e blocca l'operazione (Accesso Negato).

🔮 Step 5 (Con la `s`): Il trucco del SUID

La `s` (Set-User-ID) posizionata al posto della `x` del proprietario cambia le regole del gioco. Significa: "Chiunque riesca ad avviare questo programma, lo farà assumendo temporaneamente l'identità e i poteri del proprietario (Mario)".

🎯 Il risultato finale:

Quando il cameriere lancia `enqueue`, il programma "indossa i panni" di Mario. Il sistema operativo pensa: "Oh, è Mario che sta chiedendo di modificare `orders.txt`. Mario ha i permessi, procediamo!". E l'ordine viene aggiunto con successo.

6. Ora c'è da definire il gruppo del file `orders.txt` che in un primo momento potrebbe sembrare `mario`, infatti quando un file viene creato proprietario e gruppo coincidono.

```
- rW ____ _ mario mario orders.txt
- rWs ____ x ____ mario waiters enqueue
- rWs ____ x ____ mario cooks dequeue
```

7. Rileggendo la specifica dell'esercizio si nota che i `cooks` devono poter leggere il file `orders.txt`, e non potendolo fare lanciando la `dequeue` lo devono poter fare come gruppo associato al file, avendo inoltre il permesso di poterlo leggere.

```
- rW r ____ _ mario cooks orders.txt
- rWs ____ x ____ mario waiters enqueue
- rWs ____ x ____ mario cooks dequeue
```

Ecco la spiegazione del perché si cambia il gruppo da `mario` a `cooks` nell'ultimo passaggio. È la mossa decisiva per rispettare alla lettera i vincoli aziendali descritti nel testo.

1. Il problema dello Step 6

Se il file rimane assegnato a `mario mario` con permessi `-rw-----`, nessuno a parte Mario può aprirlo per leggerlo. I cuochi possono usare il programma `dequeue` per cancellare gli ordini (grazie al trucco della 's'), ma se provassero ad aprire normalmente il file di testo per vedere le comande, il sistema darebbe "Accesso Negato".

2. Il vincolo del testo

L'esercizio richiede esplicitamente: *"the cooks, can read and delete existing orders"*. I cuochi devono avere la capacità di leggere il contenuto.

3. La soluzione elegante (Step 7)

Come facciamo a far leggere il file solo ai cuochi senza dare loro il potere di manometterlo a mano?

- Si cambia il gruppo proprietario del file impostandolo su `cooks`.
- Si assegna al blocco centrale (quello del gruppo) il permesso di sola lettura: `r--`.

4. Perché è un sistema a tenuta stagna?

- I Cuochi:** Possono aprire il file normalmente e leggere le comande (grazie alla `r`). Se provano a modificarle a mano con un editor vengono bloccati (manca la `w`). Per cancellare in modo sicuro usano il programma `dequeue`.
- I Camerieri:** Non fanno parte del gruppo `cooks`. Il sistema li fa quindi rimbalzare nell'ultimo blocco di permessi, quello degli "Others" (Gli Altri). Lì ci sono solo tre trattini (`---`). Risultato: i camerieri non possono assolutamente né leggere né sfiorare il file.

1. Controllo degli Accessi I

Si classifichino le seguenti situazioni come esempi di politiche di controllo degli accessi mandatory, discrezionali, o la loro combinazione. Si giustifichino le risposte date.

- I meccanismi di controllo degli accessi del sistema operativo UNIX
 - Risposta
- Il caveau di una banca in cui solo i membri del consiglio di amministrazione possono entrare
 - Risposta
- L'archivio di un ospedale comune dove la cartella clinica di un paziente P può essere consultato per condurre analisi statistiche se P ha dato un permesso scritto che autorizzi questa operazione.
 - Risposta

DEFINIZIONI FONDAMENTALI:

- DAC:** IL PROPRIETARIO DELLA Risorsa DECIDE CHI PUÒ ACCEDERVI. LA GESTIONE DEI PERMESSI È PRESSIONE E A DISCREZIONE DELL'UTENTE
- MAC:** L'AUTORITÀ CENTRALE IMPONE REGOLE RIGIDE. GLI UTENTI SOTTOBONO LA REGOLA E NON POSSONO MODIFICARE I PERMESSI TRA LORO, NESSUNO PER I DOCUMENTI CHE HANNO CREATO

- DEFINIZIONALE (DAC).** IN UNA L'UTENTE CHE CREA UN FILE NE DIVENTA IL PROPRIETARIO E DECIDE A SUA DISCREZIONE A CHI ASSEGNARE I PERMESSI
- MANDATORY (MAC).** LA REGOLA È IMPOSTA DALL'AUTORITÀ CENTRALE (LA BANCA) ED È LEGATA STRETTAMENTE ALL'IDENTITÀ/NOME DELLA PERSONA.
- COMBINAZIONE.** IL SISTEMA DI BASE È MAC. IL PAZIENTE P È CONSIDERATO "PROPRIO" DEI PROPRI DATI.

7. Controllo degli Accessi II

Si consideri il seguente insieme di diritti {R,W,X,A,L,M,Own}, dove A, L e M stanno per Append, List e Modify.

- Utilizzando la sintassi del modello di Harrison-Ruzzo-Ullman, si scriva il comando per `create(s,o)`, con il quale `s` viene creata la risorsa `o` e viene dato ad `s` il solo diritto di possesso (Own).

CI SONO POCHIE ISTRUZIONI DA RICORDARE:

- "CREA LA risorsa/oggetto `o`" → `CREATE OBJECT o` → LE IF SI SCRIVONO COME: `IF [DIRITTO] E M(S,o)`
- "CREA IL DIRITTO `s`" → `CREATE SUBJECT s`
- "DADO/TRASFERISCE IL DIRITTO `X`" → `ENTER X INTO M(S,o)`
- "RENDENDO/REMOVENDO IL DIRITTO `X`" → `EXITE X FROM M(S,o)`

2) comando `CREATE(S,o)`
`CREATE OBJECT o`
`ENTER OWN INTO M(S,o)`

END

transfer.write(s_1, s_2, o), con il quale se s_1 ha il diritto di scrittura su o allora lo trasferisce (perdendolo) a s_2 .

```

COMMAND TRANSFER.WRITE( $s_1, s_2, o$ )
IF  $w \in M(s_1, o)$ 
THEN
  ENTER  $w$  INTO  $M(s_2, o)$ 
  DELETE  $w$  FROM  $M(s_1, o)$ 
END
  
```

2009 Giugno - DAC (permessi sui file)

Controllo degli Accessi

Si consideri un sistema con tre utenti: Alice, Bob e Charlie.

Alice possiede il file `alice.bat`, Bob può solo leggerlo e scriverlo, mentre Charlie può solo eseguirlo.

Charlie può solo leggere il file `bob.bat`, che è posseduto da Bob, mentre Alice lo può solo leggere e scrivere.

Charlie possiede il file `charlie.bat`; Alice lo può solo scrivere e Bob può solo eseguirlo.

Ogni file può essere letto, scritto ed eseguito dagli utenti che lo posseggono.

a. Si scriva la matrice di controllo degli accessi corrispondente a tale situazione.

► Soluzione

b. Si scriva la matrice di controllo degli accessi che si ottiene se Charlie dà ad Alice il permesso di leggere `charlie.bat` e Alice revoca a Bob il permesso di scrivere `alice.bat`.

► Soluzione

2)

	ALICE.BAT	BOB.BAT	CHARLIE.BAT
ALICE	RWX	RW	W
BOB	RW	RWX	X
CHARLIE	X	R	RWX

b)

	ALICE.BAT	BOB.BAT	CHARLIE.BAT
ALICE	RWX	RW	RW
BOB	R	RWX	X
CHARLIE	X	R	RWX

2008 Luglio - DAC (permessi sui file)

Si consideri un sistema con tre utenti: Alice, Bob e Charlie.

Alice possiede il file `alice.bat` che può essere letto sia da Bob che da Charlie.

Charlie può leggere e scrivere il file `bob.bat`, che è posseduto da Bob, ma Alice lo può solo leggere.

Solo Charlie può leggere il file `charlie.bat` che gli appartiene.

Ogni file può essere eseguito dagli utenti che lo posseggono.

a. Si scriva la matrice di controllo degli accessi corrispondente a tale situazione.

► Soluzione

b. Si scriva la matrice di controllo degli accessi che si ottiene se Charlie dà ad Alice il permesso di leggere `charlie.bat` e Alice revoca a Bob il permesso di leggere `alice.bat`.

► Soluzione

2)

	ALICE.BAT	BOB.BAT	CHARLIE.BAT
ALICE	OX	R	
BOB	R	OX	
CHARLIE	R	RW	ORX

QUI A DIFFERENZA DI PRIMA USIAMO O PER IL PROPRIO FILE IN QUANDO NEGLI ALTRI FILE CHI DEI UTENTI CHE POSSEGGONO IL FILE POSSONO SOLO ESEGUIRE.

b)

	ALICE.BAT	BOB.BAT	CHARLIE.BAT
ALICE	OX	r	R
BOB		OX	
CHARLIE	r	rw	ORX